

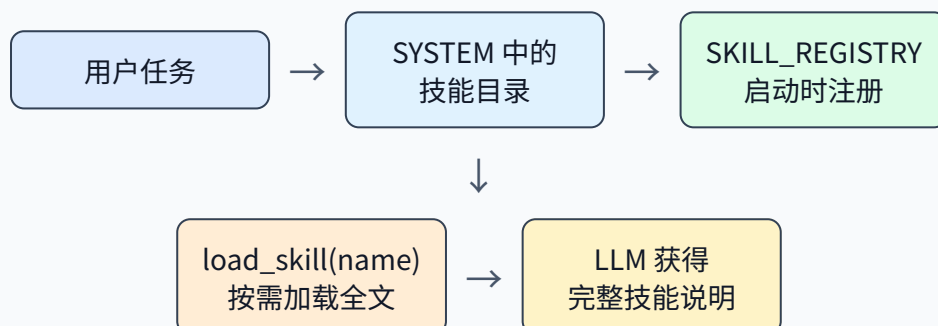
s07 Skill Loading 学习笔记

从 Skill catalog 扫描、frontmatter 解析、SYSTEM 构建，到 block.input / 工具调用 / 日志
排查

核心主线：启动时扫描技能目录，只把技能名字和描述放入 SYSTEM；真正需要某个技能时，再通过
`load_skill` 按需加载完整内容。

本次重点：tuple/list、类型注解、Path.name、dict.values、system prompt 构建、普通函数与工具函数、以及运行日志暴露的真实 bug。

总览思维导图：s07 的两层知识注入



1. s07 的整体目标：Skill catalog scan

这段代码不是在执行某一个具体技能，而是在做“技能注册表”的启动扫描。它会遍历 `skills/` 目录，读取每个子目录中的 `SKILL.md`，解析 YAML frontmatter，然后把 `name`、`description`、`content` 存入 `SKILL_REGISTRY`。

一句话：启动时只放目录，使用时再加载全文。这样可以节省 token，也让技能系统更可扩展。

```
SKILL_REGISTRY = {
    "code-review": {
        "name": "code-review",
        "description": "Perform thorough code reviews...",
        "content": "完整 SKILL.md 内容"
    }
}
```

运行流程

```
_scan_skills()
-> 遍历 SKILLS_DIR 下的子目录
-> 找到 SKILL.md
-> _parse_frontmatter(raw)
-> name = meta.get("name", d.name)
-> desc = meta.get("description", 第一行标题)
-> 写入 SKILL_REGISTRY
-> build_system() 把 catalog 注入 SYSTEM
```

2. _parse_frontmatter 为什么返回 tuple，不返回 list

函数声明是：

```
def _parse_frontmatter(text: str) -> tuple[dict, str]:
    ...
    return meta, body
```

返回值固定是两个东西：第一个是 `meta` 字典，第二个是 `body` 正文。固定数量、固定位置含义的数据更适合用 `tuple`。`list` 更像“数量可变的一组元素”。

写法	适合表达什么	例子
tuple	固定结构；位置有语义	<code>(meta, body)</code>
list	数量可变；同类元素集合	<code>["python", "git", "debug"]</code>

重点：`return meta, body` 本质上等价于 `return (meta, body)`。Python 会自动把多个返回值打包成 `tuple`。

3. tuple[dict, str] 为什么用中括号

这里的中括号不是“取下标”，也不是“创建 tuple”，而是类型注解里的泛型写法。

```
# 创建 tuple 值: 用小括号, 或者直接 return 多个值
result = ({'name': 'python'}, '正文')
return meta, body

# 标注 tuple 类型: 用中括号说明内部元素类型
result: tuple[dict, str]
```

类型注解	含义
list[str]	list 里每个元素是 str
dict[str, int]	dict 的 key 是 str, value 是 int
tuple[dict, str]	tuple 第 1 个元素是 dict, 第 2 个元素是 str

记法：变量名: 类型 = 值；参数名: 类型；函数(...) -> 返回值类型；容器类型[内部类型]。

4. 为什么要写 if len(parts) < 3

frontmatter 的标准结构是：开头一行 `---`，中间是 YAML，接着再有一行 `---`，后面才是正文。

```
---
name: python
description: Help with Python coding
---

# Python Skill
正文内容
```

执行 `text.split("---", 2)` 后，正常会得到 3 段：`parts[0]` 是开头空字符串，`parts[1]` 是 YAML，`parts[2]` 是正文。

```
parts = text.split("---", 2)

if len(parts) < 3:
    return {}, text
```

如果只有开头的 `---`，没有结束的 `---`，`parts` 只有 2 段。此时继续访问 `parts[2]` 会 `IndexError`，所以要提前防御。

高频坑：`parts` 是 list，必须写 `len(parts) < 3`，不能写 `parts < 3`。list 不能直接和 int 比较。

5. d.name、meta.get 和 description 备用逻辑

5.1 d.name 的含义

在 `for d in sorted(SKILLS_DIR.iterdir())` 中，`d` 是 `pathlib.Path` 对象，表示 `skills/` 目录下的一个子路径。`d.name` 代表路径最后一级名字。

```
from pathlib import Path

d = Path("skills/code-review")
print(d.name)      # code-review
```

因此 `name = meta.get("name", d.name)` 的意思是：优先用 `frontmatter` 里的 `name`；如果没有，就用目录名。

5.2 description 这一行

```
desc = meta.get("description", raw.split("\n")[0].lstrip("#").strip())
```

这行优先从 `meta` 字典里取 `description`；如果没有，就用 `SKILL.md` 的第一行作为描述。第一行如果是 Markdown 标题 `# Python Skill`，会先去掉左侧 `#`，再 `strip()` 去掉空格。

```
raw.split("\n")[0]      # "# Python Skill"
.lstrip("#")            # " Python Skill"
.strip()                # "Python Skill"
```

细节：`meta.get(key, default)` 的 `default` 参数会先被计算出来，即使最后没有用到。不过这里计算很轻量，一般不是问题。

6. dict.values() 和 list 的区别

普通 `dict` 有 `keys()`、`values()`、`items()`，因为 `dict` 是 `key-value` 结构。普通 `list` 没有 `values()`，因为 `list` 本身就是一组按顺序排列的值。

```
SKILL_REGISTRY = {
    "python": {"name": "python", "description": "Python skill"},
    "git": {"name": "git", "description": "Git skill"},
}

for s in SKILL_REGISTRY.values():
    print(s["name"], s["description"])
```

结构	有没有 values()	怎么遍历
dict	有	<code>for v in d.values()</code> 或 <code>for k, v in d.items()</code>
list	没有	<code>for item in lst</code>

7. SYSTEM = build_system(): 覆盖、扩充、最终字符串

SYSTEM = build_system() 是普通赋值。如果 SYSTEM 前面已经有旧值，这一行会把旧值覆盖掉。

```
SYSTEM = "old prompt"
SYSTEM = build_system() # 覆盖 old prompt
```

在 LLM API 的消息结构中，system message 的 content 通常最终要是字符串。工程上可以先用 dict/list 管理不同模块，最后统一 render 成字符串。

```
BASE_SYSTEM = f"You are a coding agent at {WORKDIR}."

SYSTEM_PARTS = {
    "base": BASE_SYSTEM,
    "skills": "",
    "rules": "Use load_skill to get full details when needed.",
}

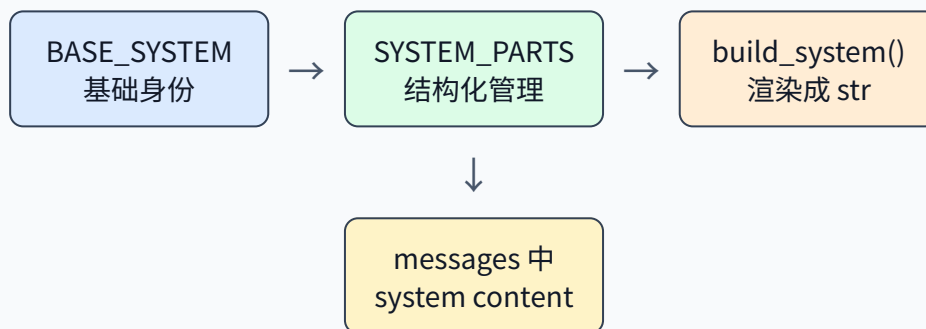
def inject_skills():
    catalog = list_skills()
    SYSTEM_PARTS["skills"] = f"Skills available:\n{catalog}"

def build_system() -> str:
    return "\n\n".join(part for part in SYSTEM_PARTS.values() if part)

inject_skills()
SYSTEM = build_system()
```

核心结论：内部可以结构化维护，最终传给模型时通常还是 str。

思维导图：SYSTEM 构建的三层



8. build_system() 为什么没有 block.input

build_system() 是程序启动阶段自己调用的普通函数，不是 LLM 通过 tool_use 调用的工具函数。它没有形参，所以只能 build_system() 空括号调用。

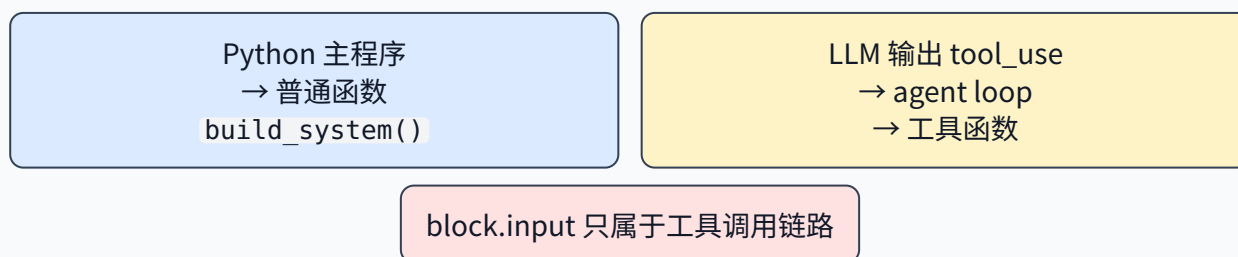
类别	例子	谁调用	参数来源
普通启动函数	<code>_scan_skills()</code> 、 <code>list_skills()</code> 、 <code>build_system()</code>	Python 程序自己调用	全局变量或显式传参
工具函数	<code>read_file(path)</code> 、 <code>write_file(path, content)</code> 、 <code>load_skill(name)</code>	LLM <code>tool_use</code> 触发 agent loop 调用	<code>block.input</code> 通过 <code>**</code> 展开

```
# 普通函数：程序自己调用
SYSTEM = build_system()

# 工具函数：模型请求 tool_use 后，agent loop 执行
result = TOOL_HANDLER[block.name](**block.input)

# 如果 block.input = {"name": "code-review"}
# 实际调用就是：
load_skill(name="code-review")
```

思维导图：普通函数 vs 工具函数



9. TypeError: can only concatenate tuple (not "str") to tuple

这个报错说明你在做 `BASE_SYSTEM + "字符串"`，但 `BASE_SYSTEM` 实际上是 tuple，不是 str。最常见原因是变量赋值后面多了一个逗号。

```
# 错误：逗号让它变成单元素 tuple
BASE_SYSTEM = (
    f"You are a coding agent at {WORKDIR}. ",
)

# 正确：去掉逗号
BASE_SYSTEM = (
    f"You are a coding agent at {WORKDIR}. "
)

# 或更简单
BASE_SYSTEM = f"You are a coding agent at {WORKDIR}. "
```

重点：真正决定 tuple 的往往不是小括号，而是逗号。("hello") 是 str，("hello",) 是 tuple。

10. 当前验证日志能看出的问题

10.1 skill 文件存在，但 SKILL_REGISTRY 没扫到

日志显示当前目录 `s07_skill_loading` 下没有 `skills/` 目录；父级 `learn-claude-code` 下才有 `skills/`，并且包含 `code-review`、`pdf`、`agent-builder`、`mcp-builder`。随后 `load_skill("code-review")` 返回 `Skill not found`，说明 `registry` 的扫描路径错了。

```
# 日志关键信号
skills/**/* -> (no matches)
No skills directory found
父级目录存在 learn-claude-code/skills/code-review
load_skill({"name": "code-review"}) -> Skill not found: code-review
```

根因： `SKILLS_DIR` 很可能写成 `WORKDIR / "skills"`，但当前 `WORKDIR` 是 `s07_skill_loading`；真正目录在 `WORKDIR.parent / "skills"`。

```
# 可修复为扫描多个候选目录
CANDIDATE_SKILL_DIRS = [
    WORKDIR / "skills",
    WORKDIR.parent / "skills",
]
```

10.2 read_file 有 workspace 限制，但 bash 可以绕过

日志里 `read_file` 读取父级 `skills/code-review/SKILL.md` 时提示 `Path escapes workspace`；但 `bash type "../skills/code-review/SKILL.md"` 却能读到。这说明 `read_file` 安全边界有效，但 `bash` 工具权限更大，能绕过同样的路径限制。

工程判断： 教学版可以接受；如果是生产级 agent，`bash` 也要做路径限制、命令白名单或隔离沙箱。

10.3 _parse_frontmatter 的确定 bug：parts < 3

验证脚本证明 `parts` 是 list，不能和 `int` 直接比较。

```
parts = "---\nname: test\n---\nbody".split("---", 2)
print(parts)          # ["", "\nname: test\n", "\nbody"]
print(len(parts))     # 3

# 错误
if parts < 3:
    ...

# 正确
if len(parts) < 3:
    ...
```

10.4 Windows 编码问题

日志中的 `_analyze.py` 第一次读取文件时触发 `UnicodeDecodeError: gbk codec`，后来改成 `encoding="utf-8"` 才成功。这说明 Windows 中文环境默认编码可能不是 UTF-8。

```
# 推荐统一写法
raw = manifest.read_text(encoding="utf-8")
content = safe_path(file_path).read_text(encoding="utf-8")
safe_path(file_path).write_text(content, encoding="utf-8")
```

10.5 python -c 多行命令在 Windows/shell 下不稳定

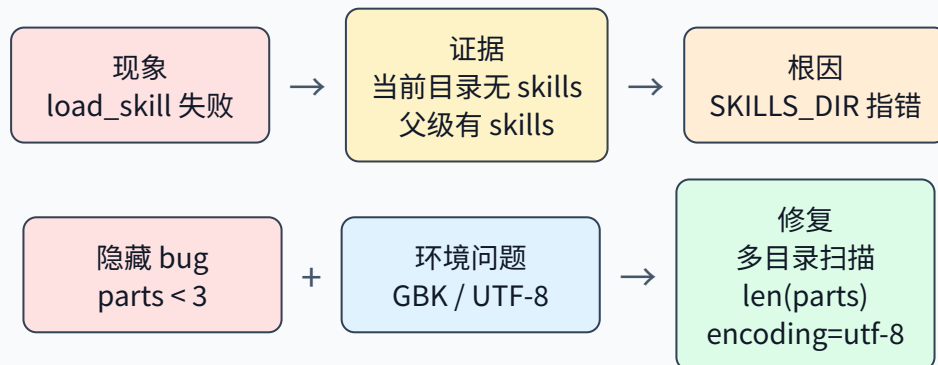
日志里多次 `python -c "...多行代码..."` 没有输出，但写入临时 `.py` 文件后能正常执行。说明复杂多行命令在 Windows + shell 混合环境里容易被引号和换行转义影响。

建议：验证逻辑较复杂时，优先写成临时 `.py` 文件再执行；不要依赖很长的 `python -c`。

10.6 ast.literal_eval 不是普通 eval

检查脚本把 `ast.literal_eval` 误报成 `eval/exec`。它不是任意代码执行，主要解析 Python 字面量，风险远低于 `eval`。

排错思维导图：日志 -> 证据 -> 根因 -> 修复



11. 推荐修复清单

优先级	问题	修复建议
P0	load_skill 找不到 code-review	修正 SKILLS_DIR，至少支持 WORKDIR / "skills" 和 WORKDIR.parent / "skills"
P0	_parse_frontmatter 中 parts < 3	改成 len(parts) < 3
P1	Windows 读取 UTF-8 文件报 UnicodeDecodeError	所有 read_text/open/write_text 明确 encoding="utf-8"
P1	bash 可绕过 read_file 的 workspace 限制	给 bash 加路径/命令限制，或明确视为高权限工具
P2	BASE_SYSTEM 误写成 tuple	去掉单元素 tuple 的尾逗号
P2	python -c 多行命令不稳定	改用临时脚本文件验证

12. 可直接套用的修正版代码片段

```
# 1) 更稳的 skill 扫描路径
WORKDIR = Path.cwd()
CANDIDATE_SKILL_DIRS = [WORKDIR / "skills", WORKDIR.parent / "skills"]

SKILL_REGISTRY: dict[str, dict] = {}

def _parse_frontmatter(text: str) -> tuple[dict, str]:
    if not text.startswith("---"):
        return {}, text
    parts = text.split("---", 2)
    if len(parts) < 3:
        return {}, text
    try:
        meta = yaml.safe_load(parts[1]) or {}
    except yaml.YAMLError:
        meta = {}
    return meta, parts[2].strip()

def _scan_skills():
    for skills_dir in CANDIDATE_SKILL_DIRS:
        if not skills_dir.exists():
            continue
        for d in sorted(skills_dir.iterdir()):
            if not d.is_dir():
                continue
            manifest = d / "SKILL.md"
            if not manifest.exists():
                continue
            raw = manifest.read_text(encoding="utf-8")
            meta, body = _parse_frontmatter(raw)
            name = meta.get("name", d.name)
            desc = meta.get("description", raw.split("\n")[0].lstrip("#").strip())
            if name in SKILL_REGISTRY:
```

```

        raise ValueError(f"Duplicate skill name: {name}")
    SKILL_REGISTRY[name] = {
        "name": name,
        "description": desc,
        "content": raw,    # 保留完整 SKILL.md；如只要正文可改成 body
    }

```

```

# 2) SYSTEM 分层构建，避免覆盖逻辑混乱
BASE_SYSTEM = f"You are a coding agent at {WORKDIR}."

def list_skills() -> str:
    if not SKILL_REGISTRY:
        return "(no skills found)"
    return "\n".join(
        f"- **{s['name']}**: {s['description']}"
        for s in SKILL_REGISTRY.values()
    )

def build_system() -> str:
    catalog = list_skills()
    return "\n\n".join([
        BASE_SYSTEM,
        f"Skills available:\n{catalog}",
        "Use load_skill to get full details when needed.",
    ])

_scan_skills()
SYSTEM = build_system()

```

13. 最终记忆卡片

知识点	一句话记忆
tuple 返回值	固定数量、固定含义的多个返回值，用 tuple 更自然。
tuple[dict, str]	类型注解：返回 tuple，里面第一个 dict，第二个 str。
len(parts) < 3	防止 frontmatter 不完整时访问 parts[2] 崩溃。
d.name	Path 对象最后一级路径名，常用作目录名 fallback。
meta.get(k, default)	字典取值；没有 key 时用默认值。
dict.values()	dict 有 value 集合；list 本身就是值集合，没有 values()。
SYSTEM = build_system()	普通赋值，会覆盖旧 SYSTEM；最终通常要生成 str。
block.input	只在 LLM 调用工具时用于给工具函数传参。
单元素 tuple	逗号才是关键：(x,) 是 tuple，(x) 不是。
日志排查	先看现象，再找证据链，再定位根因和修复点。

本次最重要结论：当前 s07 的验证日志说明，真正阻塞 skill loading 的不是 code-review 文件不存在，而是扫描路径错了；同时 `_parse_frontmatter` 里 `parts < 3` 是确定 bug，一旦路径修好并开始解析 `SKILL.md`，就可能直接触发 `TypeError`。